Session 01 — Printing and variables

Python for Everyone · Lecture notes

> **How to read the code boxes:** grey boxes with a yellow bar contain Python code. The box labelled **Output** next to it (or below it) shows what Python prints when you run that code.

What is a program?

- A **program** is a series of instructions (**commands**) that the computer carries out **one-by-one, from top to bottom**.
- A program takes some **input** (keyboard, file, internet, …), processes it, and produces some **output** (screen, file, …).
- **Python** is a programming language: a precise way of writing these instructions so that the computer understands them.

Printing

- `print(...)` displays a message on the screen.
- Text must be inside quotation marks `"..."` (or `'...'`).
- Several commands are executed in the order they appear.

```
print("Hello.")
print("Ding-dong! The Witch is dead")
```

Output:

```
Hello.
Ding-dong! The Witch is dead
```

- **Syntax** = the rules for how code must be written. Even a small deviation, like a missing quotation mark, makes Python stop with a `SyntaxError`:

```
print("Hello.)
```

Output:

```
SyntaxError: unterminated string literal (detected at line 1)
```

- Error messages are normal — everyone gets them all the time. Read them: they tell you *what* went wrong and on *which line*.

Numbers and strings

- Text in Python is called a **string** (a “string of characters”).
- Numbers are written **without** quotation marks. `print` can show several items at once, separated by commas.

```
print(1, 100, -3)
```

Output:

```
1 100 -3
```

- `1` (a number) and `"1"` (a string) look the same when printed, but Python treats them differently. You can do arithmetic with numbers:

```
print(1 + 3)
print(8 * 3 - 12)
```

Output:

```
4
12
```

- Some operators also work on strings, but they mean something different:

```
print("Hello " + "world")   # + joins (concatenates) strings
print(5 * "he")             # * with a whole number repeats a string
```

Output:

```
Hello world
hehehehehe
```

- Subtracting strings (`"Hello" - "o"`) or multiplying two strings (`"A" * "B"`) gives a `TypeError`.

Comments

- Anything after a `#` on a line is a **comment**: Python ignores it.
- Comments are notes for humans that explain what the code does.

```
# Display a friendly message
print("Don't panic!")
```

Output:

```
Don't panic!
```

- Readable code is important. It makes it easier to avoid mistakes, easier to fix them, and easier for others (or future you) to reuse the code.

Variables

- So far the values were **hard coded** (typed directly into the program). **Variables** make programs flexible.
- A variable is like a labelled bucket that holds a value (e.g. a string or a number). We refer to the value by the variable's name.
- `=` is the **assignment operator**: it puts the value on the right into the variable on the left. (It does *not* mean "equals" as in math.)
- The value of a variable can change while the program runs.

```
name = "Count Dracula"
favorite_food = "blood"
print(name + " loves " + favorite_food + ".")
name = "A mosquito"            # change the value
print(name + " loves " + favorite_food + ".")
```

Output:

```
Count Dracula loves blood.
A mosquito loves blood.
```

- `input(...)` asks the user to type something; the answer is always a **string**, which we can store in a variable:

```
name = input("What is your name?")
print("What's up,", name)
```

Naming rules (breaking them is an error):

- Only letters, digits and underscores: `a-z`, `A-Z`, `0-9`, `_`
- Cannot start with a digit: `2nd_name` is not allowed
- Case sensitive: `Name` and `name` are two different variables

Good style (not enforced, but strongly recommended):

- Short but descriptive names: `favorite_food`, not `ff` or `x2`
- Use **snake case**: lowercase words joined by underscores

Integers and floats

- There are two main types of numbers:

  - **integer** (`int`): whole numbers, e.g. `5`, `-3`
  - **float** (`float`): numbers with a fractional part, e.g. `1.65`, `0.5`
- `type(...)` tells you the type of a value.

```
x = 5
y = 7
print(type(x))
print(x / y, type(x / y))
```

Output:

```
<class 'int'>
0.7142857142857143 <class 'float'>
```

- In `x + y`, the `+` is an **operator** and `x`, `y` are its **operands**.
- `+`, `-`, `*` on two integers give an integer; division `/` **always** gives a float (`6 / 3` is `2.0`).

Operator	Meaning	Example	Result
`+`	addition	`17 + 5`	22
`-`	subtraction	`17 - 5`	12
`*`	multiplication	`17 * 5`	85
`/`	division	`17 / 5`	3.4
`//`	integer part of division	`17 // 5`	3
`%`	remainder of division	`17 % 5`	2
`**`	power	`2 ** 3`	8

Order of operations (same as in math), from first to last:

1. `( )` parentheses
2. `**`
3. `*`, `/`, `//`, `%`
4. `+`, `-`

```
print(1 + 2*3 + 10/2)
print((1 + 2)*3 + 10/2)
```

Output:

```
12.0
14.0
```

- When in doubt, use parentheses!

Changing variables

- A variable can be given a new value at any time — even of a different type.
- `x += 1` is a shorthand for `x = x + 1`. Similarly: `-=`, `*=`, `/=`.

```
x = 12
x += 1
print(x)
x = "apple"
print(x, type(x))
```

Output:

```
13
apple <class 'str'>
```

Type conversion

- `int(...)` and `float(...)` convert a value to an integer or a float.
- Converting a float to an integer **cuts off** the fractional part (no rounding): `int(12.3)` is `12`.
- Common use: turning a string into a number, so we can calculate with it.

```
s = "42"
i = int(s)
i *= 2
print(i)
```

Output:

```
84
```

- Since `input` always gives a string, convert it before doing math:

```
s = input("Please, provide a number")
f = float(s)
print("The square of", f, "is", f**2)
```

Output:

```
(typing 3)
The square of 3.0 is 9.0
```

- If the string is not a number, the conversion fails: `int("abc")` gives a `ValueError`.

Common mistakes

- **Missing quotation mark or parenthesis:** `print("Hello.)` or `print("Hello."` gives a `SyntaxError`. Every opening `"` and `(` needs a closing one.
- **Quoting a variable name:** `print("name")` prints the word *name*, while `print(name)` prints the value stored in the variable `name`.
- **Forgetting that `input` gives a string:** `age = input("Age?")` followed by `age + 1` gives a `TypeError`. Convert it first: `age = int(age)`.
- **Spelling a variable name differently:** after `name = "Ada"`, using `Name` or `nme` gives `NameError: name 'nme' is not defined`. Python does not guess what you meant.
- **Reading `=` as "equals":** `x = x + 1` is not a contradiction. It means "compute `x + 1`, then store the result in `x`".

Summary

Term / code	Meaning
`print(...)`	display values on the screen
`input(...)`	ask the user to type something; the result is a string
string (`str`)	text, written in quotes: `"Hello"`
integer (`int`)	whole number: `42`
float (`float`)	number with a fractional part: `1.65`
variable	a name that refers to a value
`=`	assignment: store a value in a variable
`#`	comment, ignored by Python
`type(...)`	the type of a value
`int(...)`, `float(...)`	convert a value to another type
`+=`, `-=`, `*=`, `/=`	update a variable, e.g. `x += 1`
`SyntaxError`	the code is not written according to Python's rules
`TypeError`	an operation used on the wrong type (e.g. `"a" - "b"`)
`ValueError`	a value that cannot be converted (e.g. `int("abc")`)